

Full Stack Deployment Using Azure (Free Tier Approach)

Overview

This project uses a Direct Deployment strategy for the frontend and a containerized backend deployment to ensure a scalable and cost-efficient architecture on Microsoft Azure. Instead of traditional CI/CD pipelines, the deployment is performed manually using CLI tools, simplifying the setup process.

Architecture Overview

Frontend: Azure Static Web Apps

Backend: Azure Container Apps

Database: Azure SQL Database (Free Tier)

Container Registry: Docker Hub

Phase 1: Frontend Deployment (Azure Static Web Apps)

Install SWA CLI

```
npm install -g @azure/static-web-apps-cli
```

Authenticate with Azure

```
swa login
```

Deploy Build Folder

```
swa deploy ./dist --env production --deployment-token <YOUR_TOKEN>
```

Phase 2: Backend Deployment Using Docker and Azure Container Apps

Dockerfile

```
FROM python:3.11-slim
```

```
ENV PYTHONDONTWRITEBYTECODE=1 \
```

```
    PYTHONUNBUFFERED=1 \
```

```
    PIP_NO_CACHE_DIR=1
```

```
WORKDIR /app
```

RUN apt-get update \

&& apt-get install -y --no-install-recommends curl gnupg unixodbc unixodbc-dev \

&& rm -rf /var/lib/apt/lists/*

RUN curl -sSL https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor -o /usr/share/keyrings/microsoft.gpg \

&& echo "deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft.gpg] https://packages.microsoft.com/debian/12/prod bookworm main" \

> /etc/apt/sources.list.d/microsoft-prod.list \

&& apt-get update \

&& ACCEPT_EULA=Y apt-get install -y --no-install-recommends msodbcsql18 \

&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --upgrade pip && pip install -r requirements.txt

COPY backend ./backend

COPY ml ./ml

COPY analysis ./analysis

COPY api ./api

COPY artifacts ./artifacts

COPY data ./data

EXPOSE 8000

CMD ["uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000"]

Push to Docker Hub

docker login

docker build -t <your-username>/<image-name>:v1 .

docker push <your-username>/<image-name>:v1

Deploy on Azure Container Apps

Create a Container App in Azure Portal.

Use Docker Hub image.

Enable ingress and set port to 8080.

Set scaling: Min replicas = 0, Max replicas = 1.

Phase 3: Database Setup (Azure SQL Database)

Create a SQL Database from Azure Portal using the free or lowest pricing tier. Configure server credentials and allow Azure services access.

Database Configuration

DB_HOST=<your-server>.database.windows.net

DB_USER=<username>

DB_PASSWORD=<password>

DB_NAME=<database-name>

Final Integration

Update frontend API URL to point to the deployed backend endpoint.

Advantages

No CI/CD complexity

Beginner-friendly

Free tier optimized

Scalable architecture

Final links

Frontend: <https://happy-cliff-0c09f4900.7.azurestaticapps.net/>

Backend: <https://backend.calmsea-8a56c76a.eastasia.azurecontainerapps.io>